# ECE 3218L Lab #1 - Introduction: Using a simulator, programmer and debugger

## Background

These laboratories will introduce you to the basic concepts of microcomputing. Microcomputing systems span a vast range, from simple single-chip processors used to control appliances to 64-bit processors with multiple cores. It is, however, somewhat reassuring that the fundamental ideas behind all of these units are about the same. Thus, by learning one microprocessor, you will be able to move to other units with relative ease.

One of the tools that you will be using in these labs is the Arduino Uno single-board microcontroller platform. It is a self-contained unit needing only a terminal for normal operation, although a host processor system for editing, assembling and downloading your programs will come in very handy, as you shall see later. The Arduino Uno boards use the AVR ATMega328P processor chip, an 8/16-bit microcontroller, as their main processor unit. We will call this the 'CPU', or the 'processor', or the 'AVR' in this document.

If you have used an Arduino before, we will not be using the standard method of programming one using the Arduino IDE and a USB port. Instead, you will attach an Arduino board to an external programmer we will refer to as the Atmel-ICE to program the microcontroller through the ICSP (in-circuit serial programming) port. The Atmel-ICE actually programs the processor -- that is, it physically downloads a program into the processor's memory. It also enables debugging of the code in situ through the development environment on which you write, debug, and test programs.

In some labs we will attach additional circuit elements to the main processor board: various switches, indicators, and integrated circuits. In later labs you will program a small robot, a Pololu 3pi that uses a processor very similar to the AVR ATMega328P. You will learn how to control the motors on the robot, to use its sensors, and to couple the two to control the robot's motion based on its environment.

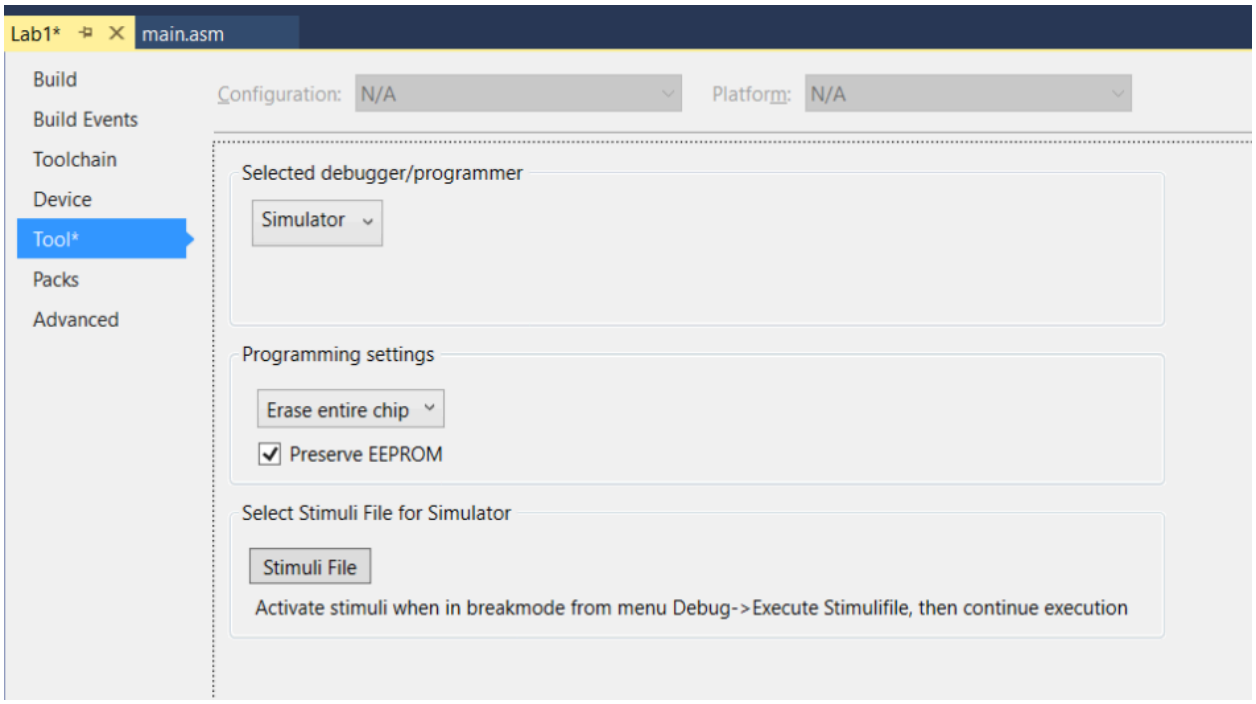
We will use an integrated development environment (IDE) called the 'Microchip Studio' from the CPU's manufacturer, Microchip (formerly Atmel). You will need to download it from the web and install it on the computer that you bring to the labs.

## Before the Lab

In the pre-labs we are going to use the AVR Simulator that is built into the Studio. A software simulator is simply a program that runs on a PC and performs the same actions as the hardware, only virtually. Its task is to act just like an AVR microcontroller. The simulator has the

same registers, and memory as the real AVR, but you get something much better. The simulator allows you to view simultaneously different areas of memory and all of the registers, all while you step through program execution. You can see the entire picture. It also simulates the IO devices (to be discussed later).

The simulator is built into the Microchip Studio IDE, and when you set up your project you have to select what tool to use when debugging and programming the project. You do this by selecting 'Debug/Lab 01 Properties', 'Tool' tab, 'Selected debugger/programmer'. Select Simulator from the drop- down menu:




This simulator will be used for all pre-labs from this point forward in the class. You will debug your programs before the lab, which will save you time and effort. The simulator is controlled using the Debug commands in the Studio – be familiar with these, as the same commands will be used when we run the code on the real hardware.

For your first microcontroller lab, you must write and assemble (generate machine language) the programs yourselves. This will familiarize you thoroughly with the instruction set and addressing modes. The first programs are reasonably simple, for your convenience. Later, the programs will get more challenging.

We will begin with a simple exercise to familiarize you with the features of the Arduino board and its simulator. Make sure that Simulator is still selected (as the 'Tool/Selected debugger/programmer').

## Read and understand the code

This routine adds five numbers stored in memory at locations 0x0100-0104.

```
; Code starts here
      .cseg
      jmp main; Jump to first real instruction
main:
      clr r0; Clear accumulator
      ldi r16,4; Set the loop counter
      ldi XH,high(data); Make X point to first byte of data
      ldi XL,low(data)
loop:
      ld r1,X+; Fetch next byte, bump index
      add r0,r1; Add byte to accumulator
      dec r16; Decrement loop counter
      brne loop; Loop if we are not done yet
      st X+,r0; Store result
fin:
      rjmp fin; Halt here

; Data starts here (On this processor this is always 0x100)
      .dseg
data:.byte 5; Space for 5 bytes
sum:.byte 1; Space for result
```

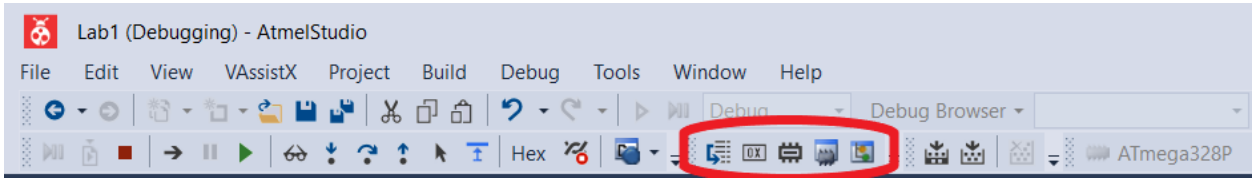
Enter this program into a newly created project and `.asm` source file. IMPORTANT –DO NOT COPY AND PASTE!! NON-PRINTING CHARACTERS MAY BE COPIED CAUSING NON-OBVIOUS ERRORS!!!

Build the program (hit F7).

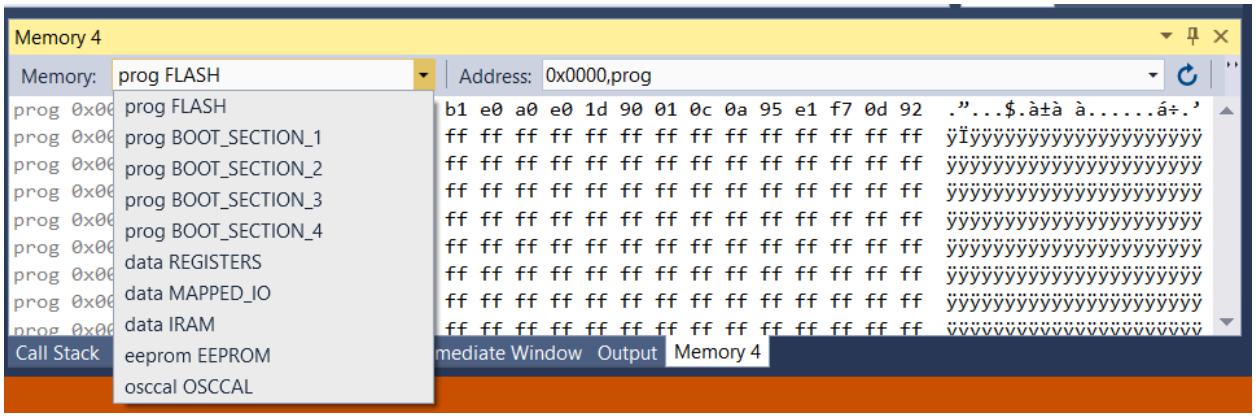
Once you have created a project with a source file and successfully *built* it, the *Debug / Start debugging and Break* command will start the simulator. When started, it will stop on the first executable instruction (PC=0). This instruction should always be a `jmp main`, where `main` is the label for the first instruction in your program. You will learn why we do this in a later lesson.

## Examining and modifying registers, memory

The simulator can show the machine registers in a window, and the contents of the various memories in another. You can access these windows from the buttons on the tool bar shown in the figure below.



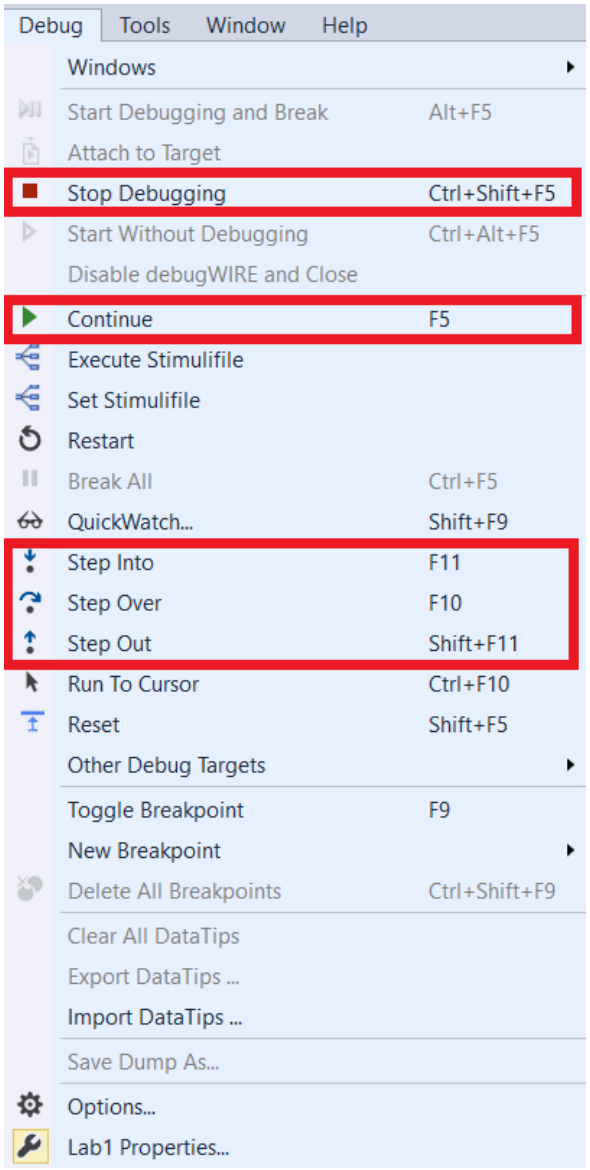

You need to select what portion of processor's memory you want to see: Program or Data or I/O memory (see the top bar of the window). Click on the register/memory cell – this is how you can modify the content of that item.




If the memory window doesn't appear as shown, step forward one instruction to initialize the contents of the memory (see below).

# Executing the code.

You can just simply run the code or step through it line by line. Study the commands under the 'Debug' menu – these are the ones you can use to control the code execution. The important commands are shown below.




## Running the program.

Stop and restart the debugger. Now enter five numbers -- try 01, 02, 03, 04 and 05 – into the data memory in locations 0x0100-0104. You can easily enter the numbers in the memory window of the development environment. Select “data IRAM” from the drop-down menu. Addresses 0x0100-0104 should be visible at the top of the window. Left click on the byte in the first location and type in the numbers. If you get a message “`Unavailable when debugger is running`” in the Memory window, press “Break All” button in the Debug menu.

Select `Debug/Continue`, then navigate in the source code to the line containing `fin: rjmp fin`. Place a breakpoint there by left clicking in the space to the left of the label. When the program is done, it will stop and you can look at the contents of the registers and the memory.

Check the contents of accumulator `r0` using the register window. Is it the sum of your five numbers? NO? Congratulations! You have inherited your first bug, courtesy of the management.

## Debugging programs (breakpoints)

We are not purposefully being mean; this is an exercise to demonstrate one of the most powerful tools used in debugging programs, the breakpoint. A breakpoint suspends execution of the program after the instruction prior to it. Then you can observe the machine state (contents of the register and memory). By comparing the actual state with what you think it ought to be, you can usually find the reason the program is not behaving as expected. In programs involving loops, it is most important to know what is going on just before entering the loop, at the critical state inside the loop, and just after exiting the loop.

Breakpoints are set by left-clicking just to the left of the label field of the instruction at which you want to stop, by selecting the *Toggle breakpoint* item from the *Debug* menu or by hitting F9. You can place a breakpoint at any executable instruction. Insert breakpoints just before entering the loop; (NOTE that breakpoints must ALWAYS be specified at locations where instructions begin!!) and just after the loop business has been done.

Now run your program again (*Stop debugging*/*Start debugging*). It will stop at the first breakpoint and tell you all about the machine state. Note that the instruction AT THE BREAKPOINT has not yet been executed. Note the machine state before you enter the loop. Is everything OK? To continue execution after encountering a breakpoint, simply hit F5, which means "go at the place pointed to by the program counter". Observe the execution of your program through the loop. **Record this in your lab notebook.** Watch each variable as the loop iterates. How many times did you go through the loop before falling out of it? By now you should have picked up the error in your program. Fix the program in the source code, re-build it, and run it for the last time to check it. Note the values in your list and the corrected sum.

You can also execute your program instruction by instruction – this is called ‘single stepping’. Start the debugging, and press F11, observe what happens.

In addition to the above write and test programs to do the following before coming to the lab:

1. A list of numbers (representing, for example, the weights of items coming off of an assembly line) begins at location 0x0100. The end of the list is signified by a “null” byte (0x00), indicating an item of 0 weight (no item). Find the total number of items produced.

2. Now write a program to find the maximum value in the list.

3. Quality control requires some degree of uniformity in the finished product. Write a program to find out whether the maximum value in the list is greater than twice the minimum value in the list.

You must make your own list; use at least 15 numbers. These numbers can be entered into the data memory by hand.

**IMPORTANT: for each of the programs above, create a flow chart or diagram to describe your program flow and execution. Record this in your lab notebook.**

# In the Lab

Have the lab TA approve the code that you have generated. You'll be taking programs you have written and running them on the Arduino board. To do this you are going to need the Atmel-ICE board. The Atmel-ICE takes the binary files and copies them to the program memory on the Arduino. It is also capable of debugging the Arduino by passing state information from the microcontroller to the computer.

## Connecting the Atmel-ICE and the Arduino

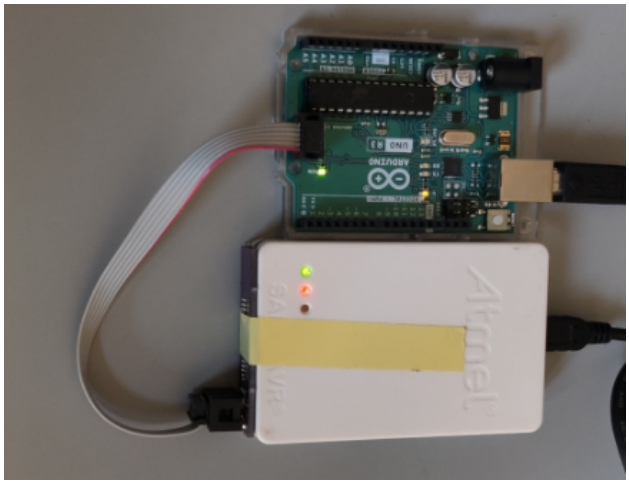
Connections need to be made as follows:

1. Connect the Arduino board to the Atmel-ICE using the 6-pin ribbon cable. This connection must be verified and approved by the TA before you proceed. Pay attention to the dark/pink cable connection of the 6-pin ribbon cable (check the picture below). There is a "key" on the ribbon cable that must face inward. ***Be careful with this cable, as the connections can be fragile.*** Always pull from the connector, never the cable.

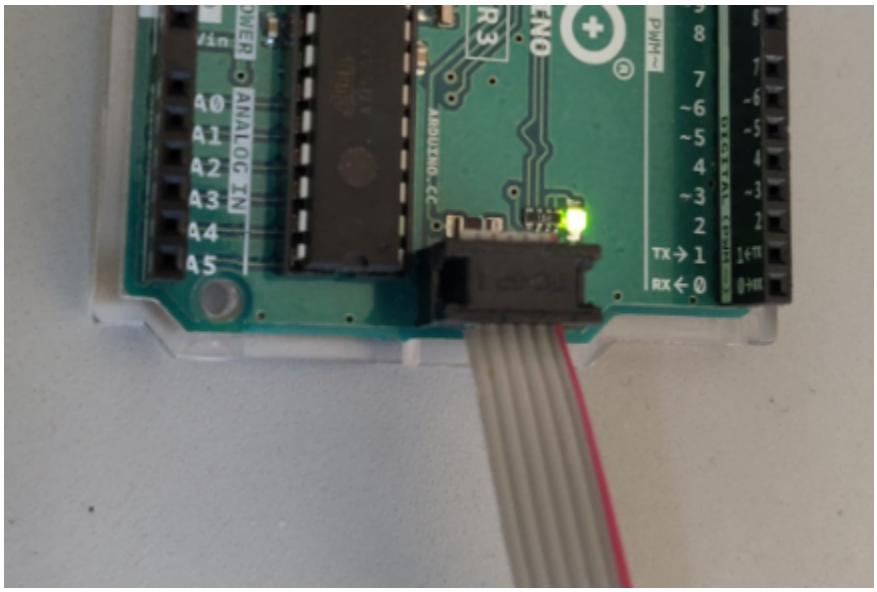
2. The Arduino Uno and Atmel-ICE need to be connected to USB. We will provide a USB hub that has on/off switches for each port. This will be quite useful for debugging. Connect the Arduino to your 4-port hub and connect the hub to your laptop's USB port.

3. Connect the Atmel-ICE to another port on the USB hub – this will be used to power and communicate with the Atmel-ICE board using the Microchip Studio.

This is what the connection should look like:



Atmel-ICE connected to the Arduino Uno.



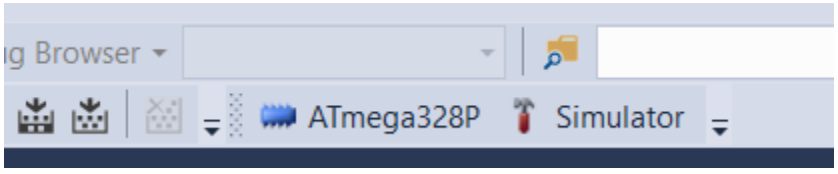

Orientation of the ICSP connector.  Note the key faces the inside of the board.



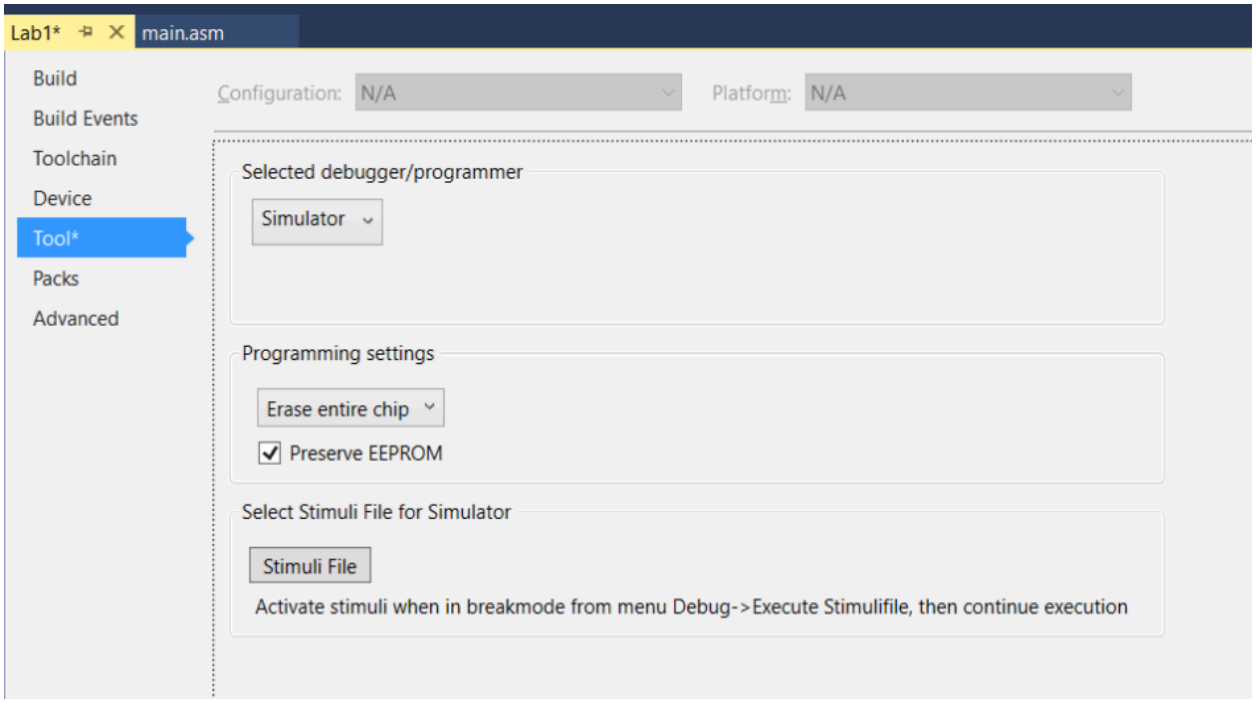

4-port USB hub with on/off switches.

## Programming the Atmel ATmega 328P in Microchip Studio

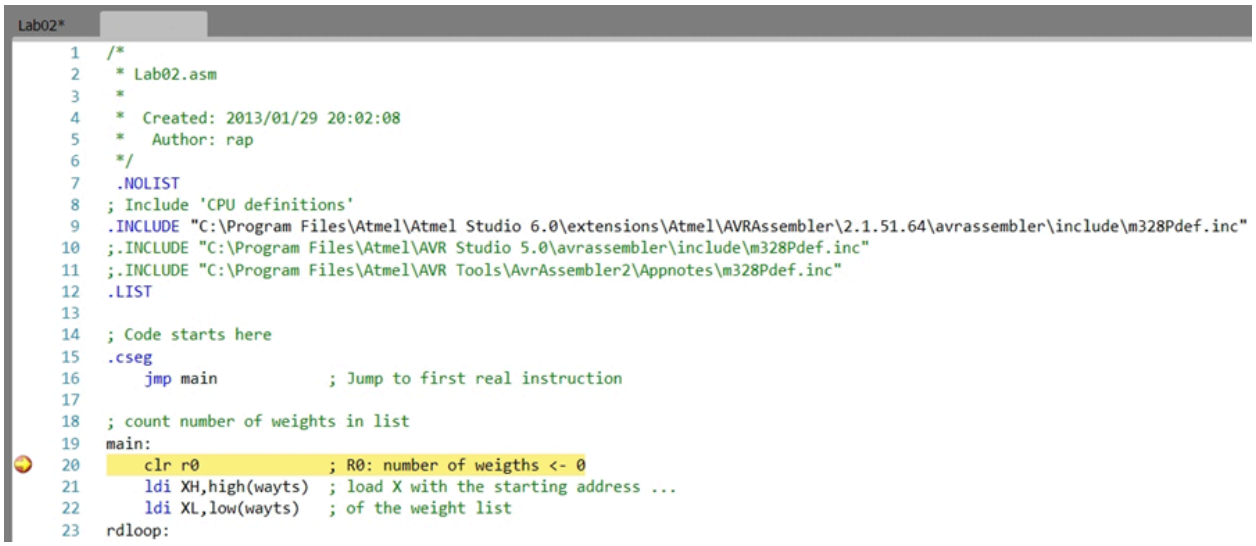
First, make sure that the rightmost two buttons on the bottom of the toolbar read `ATmega328P` and `Simulator`.




If either does not, left-click on the right button.  That will bring up the following window:

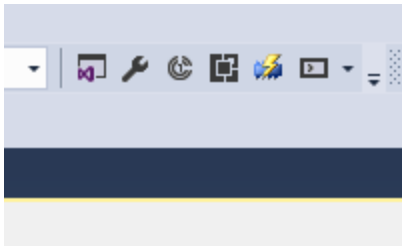



Select `Simulator` from the drop-down list. You should already have a working program from the prelab. Set a breakpoint at the first line of code, like this:

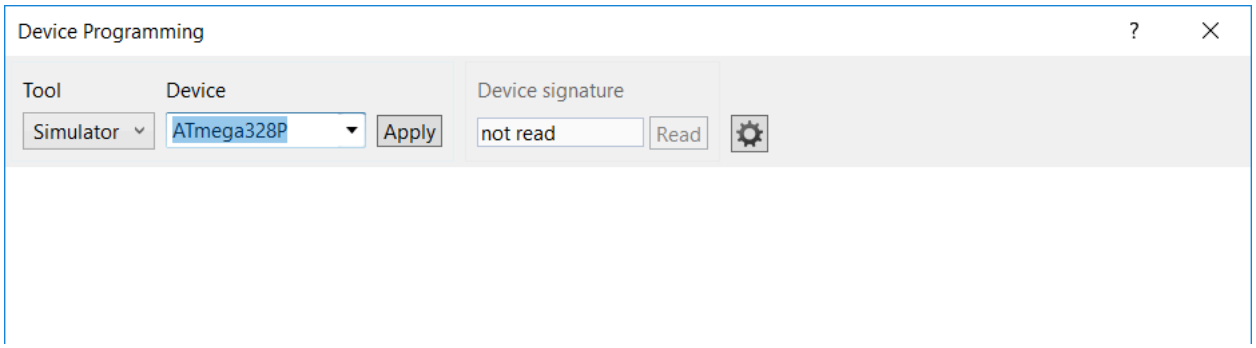



The yellow arrow will not be there initially. It is useful to set breakpoints at the end of each section of your program so that you can check the results in sequence.

Verify that the Atmel-ICE and Arduino board are correctly attached to each other with the ribbon cable and to the computer with USB Cables. Then click on the `Device Programming` button. It's the button that looks like a chip with a lightning bolt through it:



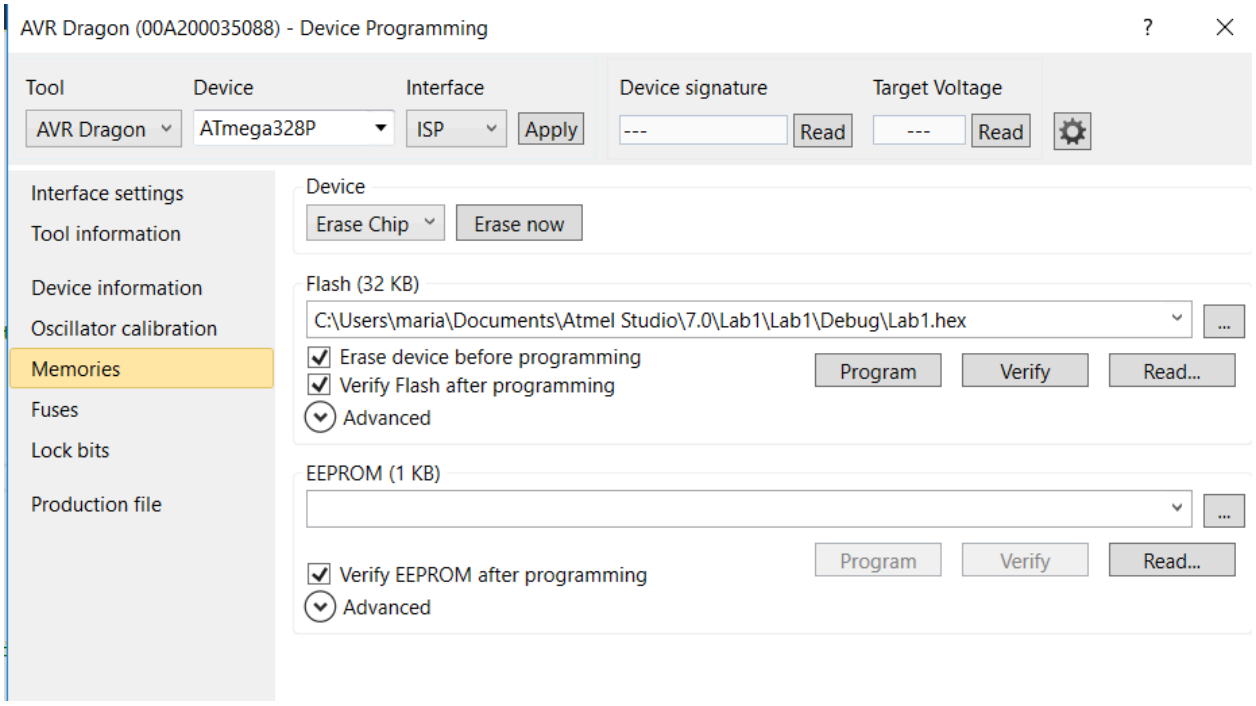
This will bring up the window:



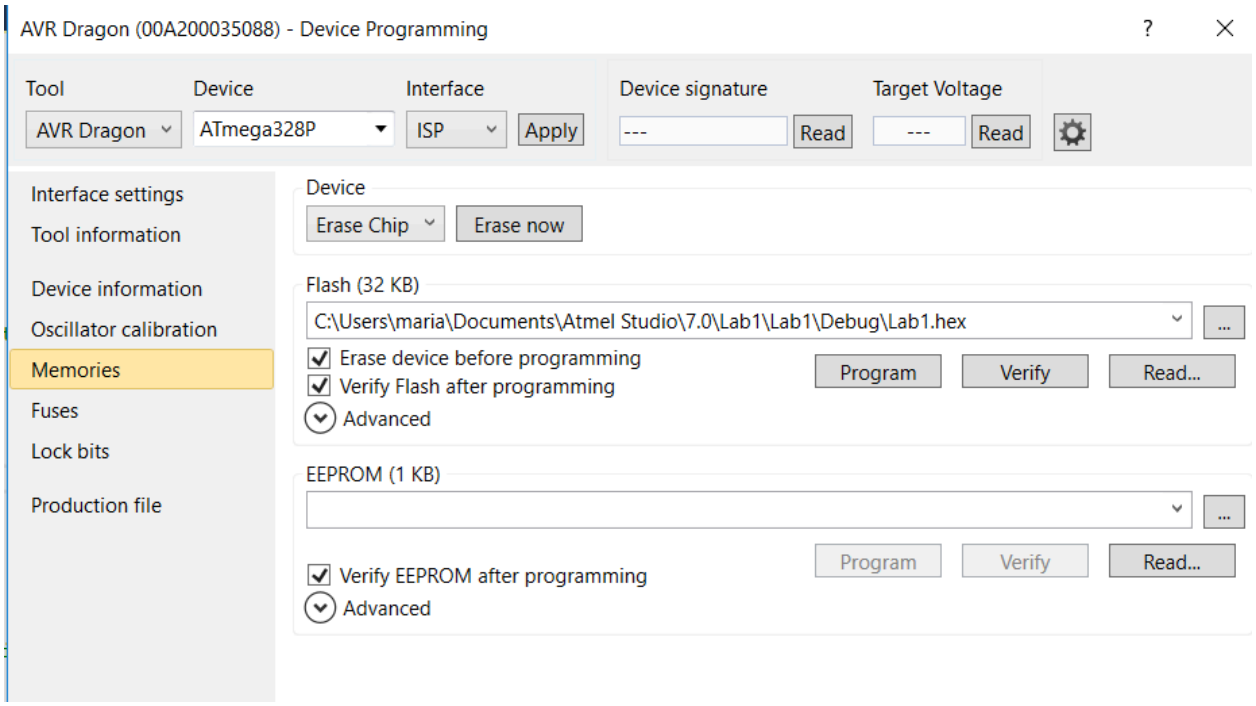

Select `Atmel-ICE` from the drop down list at the top left, select `ATmega328P` in the box directly to its right, and `ISP Interface` to the right of that. Then hit the `Apply` button. The `ISP Clock` bar should now be there. The `Device signature` box will display `not read` and the `Target Voltage` window should have `---` in it. Left click on `Read`. If you get an error message, notify the TA.

To download the program into the memory of the ATmega328P select `memories` from the list on the left.

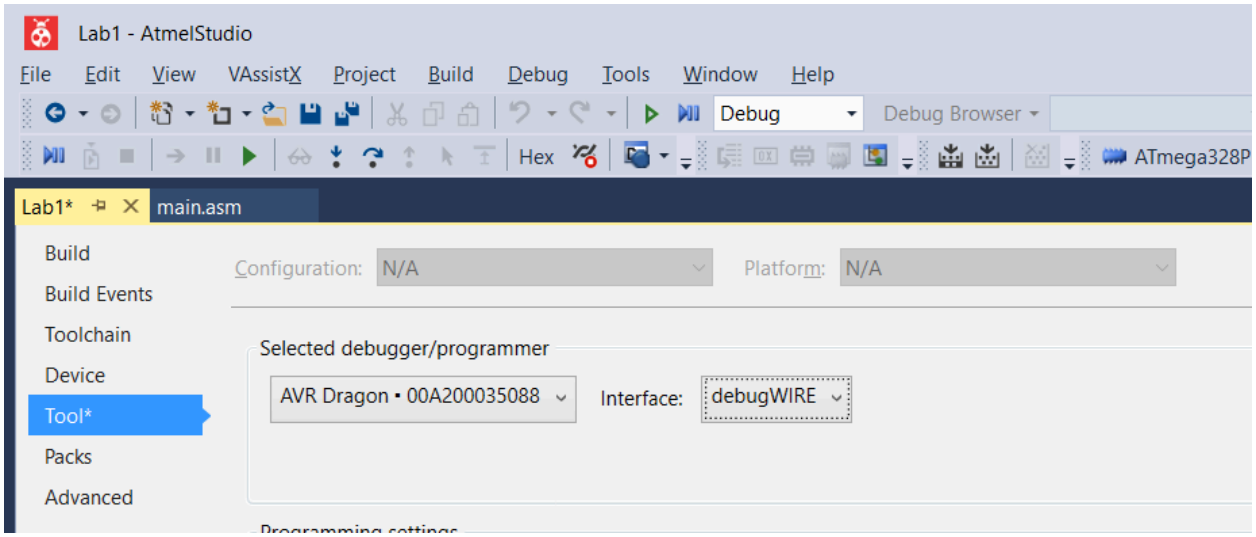
Click the `Program` button to upload your program to the Arduino board. If the program loads without a problem the box at the bottom of the window will display `Programming Flash…OK` and `Verifying Flash…OK`. The latter will not appear if the `Verify Flash after programming flag` is not set. As soon as this message shows up the program is running on the Arduino.




Now you are ready to setup to debug the program directly on the ATmega328P chip itself using debugWIRE mode. To do that, first close the device programming window, then click on the button on the toolbar that says `Simulator`:

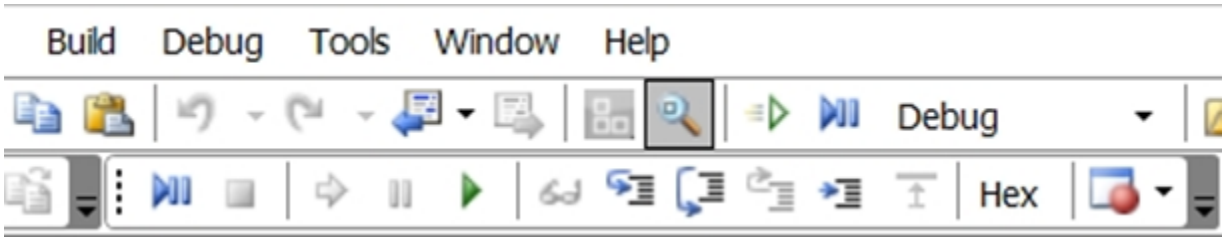


That will bring up a window that you've already seen twice before:

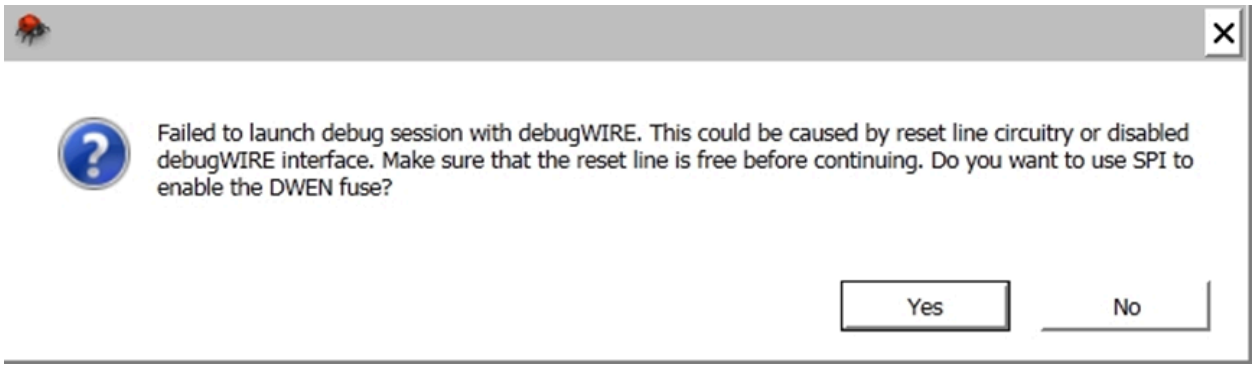



In the `Selected debugger/programmer choose` Atmel ICE and in the Interface `box`, select `debugWIRE`.

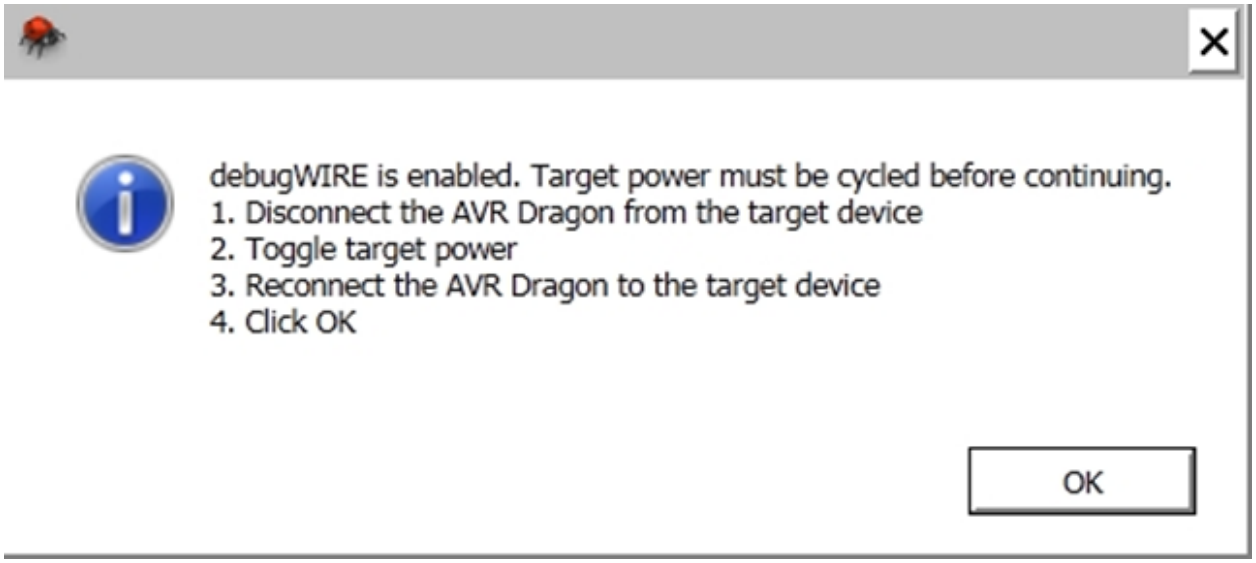
To debug the program, make sure that you have a breakpoint set at the first instruction in the main program as shown a few pages back. Then either press the `F5` key or the solid green arrow in the toolbar:




You will see the IDE setting itself up to begin debugging. If successful, it will stop at your first breakpoint. Most of the time, however, you will receive the following error dialog box:




Click on the `Yes` button. Then in a few seconds you will see something similar to this dialog box:




Use the toggle button on the USB hub port going to the Arduino Uno (not the Atmel-ICE!) to turn it off, and turn it back on again. Then hit the `OK` button. After several seconds, your program will start and stop at the first breakpoint. Now you should be able to step through your program using `F10`, which skips over subroutines, or `F11`, which steps into subroutines, or by setting breakpoints and using the `F5` key. The following is extremely important:

**To stop debugging while in the debugWIRE mode Always use Disable debugWIRE and Close. Otherwise, the device will stay in debugWIRE mode which makes it impossible to program the processor. You must always stop debugging by selecting Disable debugWIRE and Close from the Debug menu.**

If you receive an error message trying to disable debugWIRE ask your TA for help.

If you make the mistake of stopping it another way, you will discover it when you try to reprogram the processor. (It will give you an error message in a box.) You might be able to fix

the problem by restart the debugging with `F5` or the arrow button, then stop it the correct way. If that does not work, call the TA.

Not being able to read the device signature is a common problem with this setup and it can be caused by any number of problems. The most common is that the device was left in the `debugWIRE`. That can be fixed sometimes by setting a breakpoint in your program (as shown above) then hitting `F5`. If the program stops at the breakpoint (the yellow arrow will appear over the red dot) then you can fix the original problem by selecting `Disable debugWIRE and Close` from the `Debug` menu.

## Test pre-lab programs

Run all of your programs on the Arduino, using the above instructions to debug and verify that your code is running correctly. Show your TA the results of this code. Keep records of the results to use in the lab report. Test your programs with at least two different sets of values in the data lists.

## After the Lab

Produce neat, written and annotated listings of your code.

The Arduino board has a clock rate of 16.00MHz. Use your instruction manual to look up how many clock cycles each instruction in each program uses. Total the time of execution for the three programs. At $0.000001/hour of processor time, calculate how much it costs to run each of the first three programs once.

Note that the IDE can also calculate this for you. Put a breakpoint at the very end of your program and nowhere else. Start the program in Simulator running by hitting F5. When it stops (a fraction of a millisecond later) look at the “Processor Status” window, line labeled Cycle Counter. It tallied every instruction cycle that executed between from the start until the program hits the breakpoint. Compare these results to your calculation above.